Project 2

Title Page

- **Project Title:** To-do List - A Responsive Task Management Web Application
- **Submitted By:**
  - Rohan
    Roll Number: 24100110083
    Email: 24100110083.uset@ltsu.ac.in
  - Simran Kaur
    Roll Number: 24100110101
    Email: 24100110101.uset@ltsu.ac.in
  - Simranjeet Kaur
    Roll Number: 24100110102
    Email: 24100110102.uset@ltsu.ac.in
  - Simranjeet Kaur Chinti
    Roll Number: 24100110103
    Email: 24100110103.uset@ltsu.ac.in
- **Course:** Front-End UI/UX
- **Instructor Name:** Narendra Kumar Gopineedi
- **Institution:** Lamrin Tech Skills University Punjab
- **Date of Submission:** 04/01/2026

Abstract:

The To-Do List Web Application is a front-end project developed to help users manage daily tasks in a simple, organized, and efficient way. The application allows users to add tasks, mark them as completed, edit existing tasks, delete tasks, and filter tasks based on their status (all, active, or completed). The main objective of this project is to create an interactive and user-friendly task management interface using core front-end technologies.

The application is built using HTML5 for structure, CSS3 and Bootstrap for styling and responsive layout, and JavaScript for handling application logic, DOM manipulation, and data persistence. Local Storage is used to save tasks, ensuring that user data remains available even after refreshing the page. As a result, this project provides a practical demonstration of real-world front-end development concepts.

Objectives

1. To design a simple and user-friendly task management application.
2. To allow users to add, edit, and delete tasks.
3. To implement task completion functionality using checkboxes.
4. To provide filtering options for active and completed tasks.
5. To store tasks using browser local storage for data persistence.
6. To enhance front-end development skills using HTML, CSS, JavaScript, and Bootstrap.

Scope of the Project

1. The project focuses entirely on front-end development.
2. No backend server or database is used.
3. Task data is stored locally using the browser's Local Storage.
4. The application is responsive and works on desktop and mobile screens.
5. Only open-source tools and standard web technologies are used.
6. The project is intended for learning and demonstrating core JavaScript and UI concepts.

Tools & Technologies Used

Tool / Technology	Purpose
HTML5	Defines the structure and layout of the web application
CSS3	Handles styling, layout design, and visual appearance
Bootstrap 5	Provides responsive design and reusable UI components
JavaScript	Implements application logic and user interactions
Local Storage	Stores tasks locally for data persistence
Visual Studio Code	Used for writing and managing source code
Chrome DevTools	Used for testing, debugging, and performance analysis

HTML Structure Overview

1. The HTML file is organized around a main container that holds the entire to-do list application.
2. A heading is used to display the title of the application at the top.
3. An input field and add button allow users to enter new tasks.
4. Filter buttons (All, Active, Completed) are provided to manage task views.
5. An unordered list dynamically displays the to-do items.
6. Each task item includes a checkbox, task text, and a delete button.

CSS Styling Strategy

1. **Bootstrap Integration:** Bootstrap is used to center the application, apply spacing, and ensure responsiveness across devices.
2. **Custom Styling:** Custom CSS enhances the appearance of the container, input fields, buttons, and task items.
3. **Gradient Background:** A linear gradient background is applied to create a modern and visually appealing look.

4. **Hover Effects:** Buttons include hover effects for better user interaction feedback.
5. **Clean UI:** Simple colours, rounded corners, and readable fonts provide a clean and minimal interface.

Key Features

Feature	Description
Add Tasks	Allows users to add new tasks using the input field
Edit Tasks	Enables users to edit tasks by double-clicking on the task text
Delete Tasks	Provides an option to remove tasks from the list
Task Completion	Allows users to mark tasks as completed using a checkbox
Task Filters	Enables filtering of tasks as All, Active, or Completed
Local Storage	Stores tasks in the browser to retain data after page refresh
Responsive Design	Ensures smooth performance on both desktop and mobile devices
User-Friendly Interface	Simple and clean layout with easy interactions

Challenges Faced & Solutions

Challenge	Solution
Managing task state	Used JavaScript objects and arrays to store and manage task data
Data persistence	Implemented Local Storage to save tasks across page refreshes
Filtering tasks	Applied conditional logic to filter tasks based on their status
Updating the DOM	Used a centralized render function to update the UI consistently
UI responsiveness	Utilized Bootstrap layout and flexible CSS styling

Outcome

1. A clean, functional, and user-friendly to-do list web application was successfully developed.
2. The application supports essential task management features such as adding, editing, deleting, and filtering tasks.
3. Tasks are stored persistently using browser Local Storage, ensuring data remains after page refresh.
4. The project improved understanding of DOM manipulation and event handling in JavaScript.
5. Practical experience was gained in implementing real-world JavaScript logic for interactive applications.

Future Enhancements

1. Add due dates and priority levels to tasks for better task organization and planning.
2. Implement drag-and-drop functionality to allow easy reordering of tasks.
3. Introduce a dark mode toggle to improve usability in low-light environments.
4. Add task counters to display the number of active and completed tasks.
5. Integrate a backend service to enable task synchronization across multiple devices

Sample Code

HTML: This section creates a centered, shadowed box that holds a title and a text box for typing new tasks. It features an "Add" button to save tasks and a row of filter buttons to switch between viewing all, active, or completed items. All tasks are displayed in an empty list that updates automatically, with the app's behavior controlled by a linked script file at the bottom.

```
<body>
    <div class="container d-flex justify-content-center align-items-start mt-5">
        <div class="todo-container shadow p-4 rounded">
            <h2 class="text-center mb-4">To-Do List</h2>
            <div class="input-group mb-3"> <input type="text" id="todo-input" class="form-control"
                    placeholder="Enter a task"> <button class="btn btn-primary" id="add-btn">Add</button> </div>
            <!-- Filter Buttons -->
            <div class="btn-group mb-3 w-100"> <button class="btn btn-outline-secondary filter-btn"
                    data-filter="all">All</button> <button class="btn btn-outline-secondary filter-btn"
                    data-filter="active">Active</button> <button class="btn btn-outline-secondary filter-btn"
                    data-filter="completed">Completed</button> </div>
            <ul id="todo-list" class="list-group"></ul>
        </div>
    </div>
    <script src="script.js"></script>
</body>
```

CSS: This stylesheet defines the look of the app by giving the main container a fixed width, rounded corners, and a light shadow for a modern card effect. It centers the heading and uses "flexbox" to neatly align the input field and button with consistent spacing. The design focuses on clean borders and simple colors to make the interface user-friendly.

```css
/* Main container */
.todo-container {
    background: #ffffff;
    width: 420px;
    padding: 20px;
    border-radius: 8px;
    box-shadow: 0 3px 10px rgba(0,0,0,0.1);
}

/* Heading on top */
.todo-container h1 {
    text-align: center;
    margin-bottom: 15px;
    font-size: 22px;
    color: #222;
}

/* Input section BELOW heading */
.input-group {
    display: flex;
    gap: 10px;
    margin-bottom: 15px;
}

/* Input */
#todo-input {
    flex: 1;
    padding: 8px 10px;
    border-radius: 5px;
    border: 1px solid #ccc;
    font-size: 14px;
}
```

JavaScript: This script handles the app's interactivity by creating task elements and adding them to the list. It includes logic to cross out completed tasks, a "double-click" feature to edit existing text, and a delete function to remove items. Finally, it ensures that all changes are saved and the display is updated automatically whenever a task is modified.

```javascript
    // Text of the Todo
    const textSpan = document.createElement("span");
    textSpan.textContent = todo.text;
    textSpan.style.margin = '0 8px';
    // textSpan.classList.toggle('completed', todo.completed);

    if(todo.completed){
        textSpan.style.textDecoration = 'line-through';
    }
        // Add double click event listner to edit todo
        textSpan.addEventListener("dblclick", ()=>{
            const newText = prompt("Edit todo", todo.text);
            if(newText !== null){
                todo.text = newText.trim()
                textSpan.textContent = todo.text;
                saveTodos();
            }
        })

        // Delete Todo button
        const delBtn = document.createElement('button');
        delBtn.textContent = "Delete";
        delBtn.addEventListener('click', ()=>{
            todos.splice(index, 1);
            render();
            saveTodos();
        })

        li.appendChild(checkbox);
        li.appendChild(textSpan);
        li.appendChild(delBtn);
        return li

}
```

Screenshot of Final Output

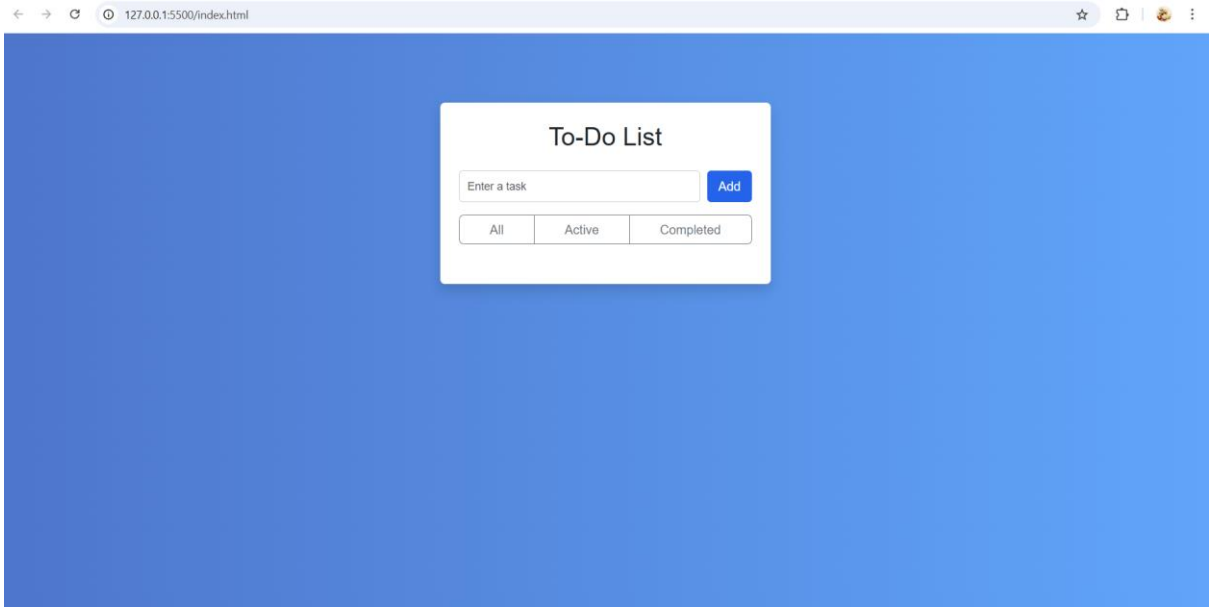

Figure 1: A clean and minimalist **To-Do List interface** featuring a central white card with an input field, an "Add" button, and status filters for "All," "Active," and "Completed" tasks.

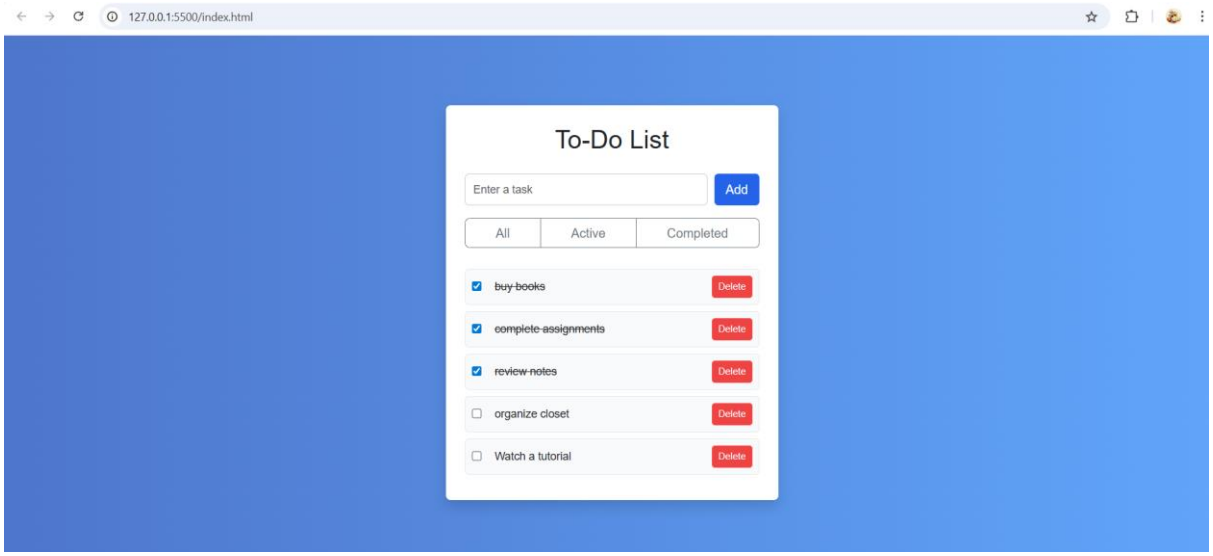

Figure 2: A view of the **To-Do List** showing several tasks marked as finished with blue checkmarks and lines through the text. The design features a simple white box on a blue background, allowing users to easily see which chores are done and which are still active.

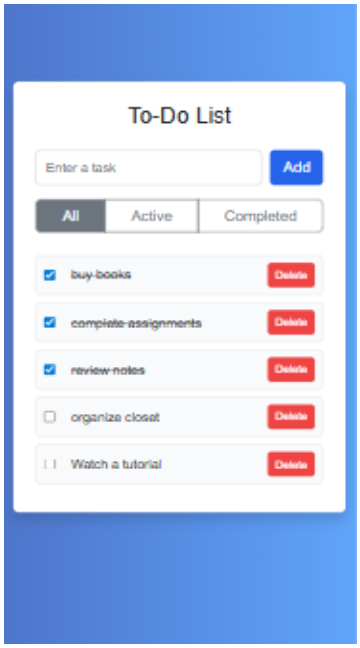

Figure 3: A **mobile-friendly To-Do List** layout with a simple, vertical design that is easy to scroll through on a phone. It features a compact top section for adding tasks and large, easy-to-tap buttons for checking off or deleting items.

Conclusion

This project served as a practical exercise in developing a functional and responsive front-end web application using core web technologies. The To-Do List Web Application effectively demonstrates the use of JavaScript for DOM manipulation, event handling, task filtering, and data persistence through Local Storage. Working on this project enhanced problem-solving abilities and provided valuable hands-on experience with real-world front-end development practices. The final application is a simple yet efficient task management tool that reflects a strong understanding of user interface design and JavaScript fundamentals.

Reference

1. **MDN Web Docs – JavaScript** – https://developer.mozilla.org/enUS/docs/Web/JavaScript
Referenced for JavaScript functions, async/await, fetch API, and DOM manipulation.

2. **MDN Web Docs – CSS** – https://developer.mozilla.org/en-US/docs/Web/CSS
Used for styling cards, hover effects, transitions, and responsive layouts.

3. **Bootstrap Documentation** –https://getbootstrap.com/docs/5.0/gettingstarted/introduction/
Used for grid layout, responsiveness, and prebuilt components.

4. L&T LMS: https://learn.lntedutech.com/Landing/MyCourse

5. Repo: https://github.com/Simran-317/To-do-Application

6. Site: https://simran-317.github.io/To-do-Application/